

WorkTrace — Technical Reference

A component-by-component breakdown of how WorkTrace works under the hood.

Table of Contents

- 1. [Service topology](#)
- 2. [Data models](#)
- 3. [Configuration layer](#)
- 4. [Authentication](#)
- 5. [Ingestion pipeline](#)
- 6. [Embedding layer](#)
- 7. [Semantic search](#)
- 8. [RAG chat pipeline](#)
- 9. [LLM provider abstraction](#)
- 10. [Meeting intelligence pipeline](#)
- 11. [Time Task Tracker — write path](#)
- 12. [Time Task Tracker — read path \(chat context\)](#)
- 13. [TTT REST API](#)
- 14. [Frontend architecture](#)
- 15. [OpenShift deployment](#)
- 16. [File reference](#)

1. Service topology

Local development

Service	Runtime	Port	Role
PostgreSQL + pgvector	Docker (docker-compose.yml)	5433	Document chunks, 768-dim vectors, metadata
Ollama	Native	11434	Local embedding (nomic-embed-text) + local LLM (llama3.2)
FastAPI / uvicorn	Python	8000	REST API — all backend logic
Vite / React	Node	5173	Frontend SPA

OpenShift (production)

```
Browser → OpenShift Route (HTTPS)
  → frontend pod (nginx :8080)
    → serves static Vite build
    → proxies /api/* → backend ClusterIP :8000
      → FastAPI pod
        → in-cluster pgvector (Ceph RBD PVC, port 5
        → Supabase (auth only, external)
        → Nomic Atlas (embeddings, external)
        → watsonx / Groq (LLM, external)
```

The backend is never publicly exposed — only the frontend Route is. All API calls go through nginx inside the cluster. `VITE_API_URL=/api` is baked into the frontend image at build time so every fetch goes to the relative `/api` path, which nginx proxies to the backend ClusterIP service.

The separate TTT database (`TTT_DATABASE_URL`) is either the same Supabase/Neon project or a separate one. The main vector database is in-cluster pgvector.

2. Data models

documents table (pgvector DB)

Defined in [backend/kb/db.py](#).

Column	Type	Description
id	INTEGER PK	Auto-increment
source	VARCHAR(1024)	Original filename or path — the dedup key
file_type	VARCHAR(64)	pdf , text , docx , image , generic
chunk_index	INTEGER	Position of this chunk within the source (0-based)
content	TEXT	Raw text of the chunk
embedding	Vector(768)	pgvector column — cosine similarity target
created_at	TIMESTAMP	Insert time
user_id	VARCHAR	Supabase user UUID — all queries are scoped to this

Column	Type	Description
doc_metadata	JSONB	Parsed meeting header fields: meeting_date, meeting_title, organizer, attendees, platform, meeting_time, project_code, billable, duration_minutes

doc_metadata and user_id are added via ALTER TABLE ... ADD COLUMN IF NOT EXISTS in init_db() so older deployments without them are migrated transparently on startup. An index on source makes dedup checks and source-filter queries fast.

time_entries table (TTT DB)

Written to by [backend/kb/pusher.py](#) and managed by [backend/ttt_api.py](#). Read at chat time by [backend/kb/ttt.py](#).

Column	Type	Description
id	TEXT PK	UUID string
user_id	TEXT	Supabase user UUID — all queries are scoped to this
project_code	TEXT	Project identifier
task_type	TEXT	meeting for KB-pushed entries; any string for manual entries
duration_minutes	NUMERIC	Duration in minutes
entry_date	DATE	Date of the entry
start_time	TIMESTAMPTZ	Optional start time (naive, no tz conversion)
end_time	TIMESTAMPTZ	Optional end time (naive, no tz conversion)
description	TEXT	LLM-generated summary or user-provided notes
meeting_title	TEXT	Meeting title
billable	BOOLEAN	Whether the entry is billable
confidence	NUMERIC	Confidence score (0.75 for KB-pushed; 0.0 for manual)
status	TEXT	logged (default)

Column	Type	Description
organizer	TEXT	Meeting organizer
attendees	TEXT	Comma-separated attendee list

3. Configuration layer

File: [backend/kb/config.py](#)

All configuration is environment-driven. `config.py` calls `load_dotenv()` against candidate paths in priority order:

1. `KB_ENV_FILE` env var (explicit override)
2. `./env` in the current working directory
3. `.env` in the project root

Every other module imports constants directly from `config.py`. Nothing reads `os.environ` elsewhere.

Environment variable reference

Variable	Description
<code>DATABASE_URL</code>	pgvector PostgreSQL connection string
<code>TTT_DATABASE_URL</code>	Separate TTT PostgreSQL connection string
<code>SUPABASE_URL</code>	Supabase project URL (used to fetch JWKS)
<code>SUPABASE_JWT_SECRET</code>	Static JWT secret (HS256 fallback)
<code>EMBED_PROVIDER</code>	<code>nomic</code> or <code>ollama</code>
<code>NOMIC_API_KEY</code>	Required when <code>EMBED_PROVIDER=nomic</code>
<code>EMBED_DIMENSIONS</code>	Vector dimension — must match the embedding model (default 768)
<code>LLM_PROVIDER</code>	<code>ollama</code> , <code>openai</code> , or <code>watsonx</code>
<code>OLLAMA_HOST</code>	Ollama base URL (default <code>http://localhost:11434</code>)
<code>OLLAMA_CHAT_MODEL</code>	Ollama chat model name
<code>OLLAMA_EMBED_MODEL</code>	Ollama embedding model name

Variable	Description
OPENAI_API_KEY	API key for any OpenAI-spec provider
OPENAI_BASE_URL	Base URL override (e.g. Groq, ZhipuAI)
OPENAI_CHAT_MODEL	Model name
WATSONX_API_KEY	IBM Cloud API key
WATSONX_PROJECT_ID	watsonx.ai project ID
WATSONX_MODEL_ID	watsonx.ai model ID
WATSONX_URL	watsonx.ai inference URL
RAG_TOP_K	Default number of chunks retrieved per search

Environment variable precedence (OpenShift)

```

OpenShift Secret (via deploy.sh)      ← highest
  ↓
deploy.env / render.yaml envVars
  ↓
.env file (local only)
  ↓
config.py defaults                    ← lowest

```

4. Authentication

File: [backend/kb/auth.py](#)

All routes that read or write user data use `get_current_user` as a FastAPI dependency. It validates the Supabase JWT from the `Authorization: Bearer` header and returns the user's UUID (`sub` claim).

Token validation

Supabase projects may sign JWTs with either HS256 (static secret, older projects) or ES256 (rotating asymmetric key from JWKS, newer projects). The validator handles both:

1. **ES256 via JWKS** — on first call, fetches `{SUPABASE_URL}/auth/v1/.well-known/jwks.json` and caches key objects by `kid`. Each incoming token's `kid` is looked

up in the cache; if not found the JWKS is re-fetched once before failing over to HS256.

2. **HS256 static secret** — falls back to `SUPABASE_JWT_SECRET` if JWKS is unavailable or the token's `kid` is not found.

The JWKS key cache is protected by a `threading.Lock` — safe for concurrent requests.

User isolation

Every database query in the ingestion, search, chat, and TTT paths includes a `WHERE user_id = :user_id` clause (or equivalent ORM filter) using the UUID returned by `get_current_user`. Users cannot read or modify other users' documents or time entries.

5. Ingestion pipeline

Files: [backend/kb/ingest.py](#) · [backend/kb/extractors.py](#)

Entry points:

- **API:** `POST /upload` (multipart) and `POST /ingest-meeting` (multipart)
- **CLI:** `kb ingest <path>`

Both converge on `ingest(path, force=False, source_name=None, user_id=None)`.

Step 1 — File discovery

`_iter_files(root)` walks the path. A single file is yielded directly. A directory recursively yields all files whose suffix is in `SUPPORTED_SUFFIXES`.

Step 2 — Dedup check

`_already_indexed(session, source_key, user_id)` checks for an existing row with the same `source` and `user_id`. If found and `force=False`, the file is skipped. If `force=True`, all existing rows for that source and user are deleted before re-ingesting. The `source_key` is the original upload filename — the DB key is always the human-readable name, not a temp path.

Step 3 — Extraction and chunking

`extract(path)` in [extractors.py](#) dispatches by file extension:

Suffix	Extractor	Notes
<code>.pdf</code>	<code>extract_pdf</code>	pypdf — concatenates all page text
<code>.docx</code> , <code>.doc</code>	<code>extract_docx</code>	python-docx — joins paragraph text
<code>.png</code> , <code>.jpg</code> , <code>.jpeg</code> , <code>.gif</code> , <code>.bmp</code> , <code>.tiff</code> , <code>.webp</code>	<code>extract_image</code>	Pillow + pytesseract OCR
<code>.txt</code> , <code>.md</code> , <code>.csv</code> , <code>.json</code> , <code>.yaml</code> , <code>.html</code> , <code>.py</code> , <code>.js</code> , <code>.ts</code> , <code>.go</code> , etc.	<code>extract_txt</code>	Plain read + meeting metadata extraction
anything else	<code>extract_generic</code>	Strips non-ASCII bytes, extracts printable characters

All extractors funnel through `_chunk_text(text, size=800, overlap=100)` — fixed-size character chunks with 100-character overlap to preserve context at boundaries.

Meeting metadata extraction

`extract_txt` also calls `extract_meeting_metadata(raw)` which scans for structured header fields:

Header	Stored as
Date: ...	<code>meeting_date</code> (normalised to ISO-8601)
Meeting Title: ...	<code>meeting_title</code>
Time: ...	<code>meeting_time</code> (raw string, e.g. <code>2:00 PM – 2:45 PM CST</code>)
Organizer: ...	<code>organizer</code>
Attendees: ...	<code>attendees</code>
Platform: ...	<code>platform</code>
Project: ... / Project Code: ...	<code>project_code</code>
Billable: ...	<code>billable</code> (normalised to <code>bool</code>)

Header	Stored as
Duration: ... / Meeting Duration: ...	duration_minutes (normalised to float)

Each chunk is prefixed with a short [Meeting: ... | Date: ... | Organizer: ...] header so every chunk carries context about its source meeting even when retrieved in isolation.

Step 4 — Embedding

`embed(chunk)` is called synchronously for each chunk, returning a `list[float]` of length `EMBED_DIMENSIONS` (768).

Step 5 — Database write

A `Document` ORM row (with `user_id`) is added to the SQLAlchemy session for each chunk. The session commits after all chunks of a single file are processed. A failure rolls back only that file.

6. Embedding layer

File: [backend/kb/embedder.py](#)

```
embed(text: str) -> list[float]
embed_batch(texts: list[str]) -> list[list[float]]
```

Both delegate to the provider returned by `get_embedder()` which reads `EMBED_PROVIDER`.

OllamaEmbedder

Calls Ollama's `/api/embed` via `httpx` with `Connection: close` to force HTTP/1.1 and avoid connection-reuse issues. Returns `embeddings[0]`.

NomicEmbedder

Calls the Nomic Atlas API at `https://api-atlas.nomic.ai/v1/embedding/text` with `task_type: "search_document"`. Both providers output 768-dimensional vectors matching the `Vector(768)` pgvector column.

7. Semantic search

File: [backend/kb/search.py](#)

`search(query, top_k, file_type, source_filter, user_id)`

1. Embeds the query with `embed(query)`
2. Runs a SQLAlchemy query using pgvector's cosine distance operator `<=>` :

```
SELECT *, embedding <=> :query_vector AS distance
FROM documents
WHERE user_id = :user_id
[AND file_type = :file_type]
[AND source ILIKE '%source_filter%']
ORDER BY distance
LIMIT :top_k
```

3. Converts distance to similarity score: `score = 1.0 - distance`
4. For each result, calls `_extract_snippet(content, query)` for a short excerpt centred on the best-matching region

Snippet extraction

`_extract_snippet` uses a sliding window algorithm:

- Tokenises the query into non-trivial terms (≥ 3 chars, minus stop words)
- Finds all character positions where each term appears in the chunk
- Slides over match positions to find the window covering the most distinct query terms
- Trims to sentence or word boundaries and adds ellipsis markers

Supporting queries

Function	Purpose
<code>get_most_recent_meeting_date(user_id)</code>	Returns the latest <code>doc_metadata->'meeting_date'</code> for temporal re-ranking in chat
<code>list_sources(user_id)</code>	<code>GROUP BY source, file_type</code> with chunk count
<code>delete_source(source, user_id)</code>	Deletes all rows matching <code>source</code> and <code>user_id</code>

8. RAG chat pipeline

File: [backend/kb/chat.py](#)

Entry point: `ask(question, history, top_k, source_filter, file_type, skip_ttt, user_id)`

Step 1 — Intent classification (pre-LLM, regex)

`_is_temporal_meeting_query(question)` — matches "last meeting", "most recent standup", "latest sync", etc. When true, triggers meeting date re-ranking and forces TTT meeting history lookup.

`is_ttt_query(question)` (from `ttt.py`) — matches "hours logged", "billable", "time entries", "what did I work on", etc. When true, fetches structured TTT rows.

Step 2 — Vector retrieval

Calls `search(question, top_k, file_type, source_filter, user_id)`. Returns up to `top_k` `SearchResult` objects ordered by cosine similarity.

Step 3 — Temporal meeting re-ranking

If `_is_temporal_meeting_query` fired:

1. `get_most_recent_meeting_date(user_id)` fetches the latest `meeting_date` in `doc_metadata`
2. Retrieved chunks are partitioned: chunks from that date first, all others after
3. If no chunks from the most recent date are in the top-K, a second search runs with `top_k * 2` and the same partitioning applies

Step 4 — TTT context injection

If `skip_ttt=False` and TTT or temporal intent was detected, `query_ttt(question, user_id)` fetches structured rows from the TTT database and prepends them as a text block to the vector context — structured data before unstructured prose. `skip_ttt=True` is passed from `/summarize-meeting` to prevent historical entries from contaminating the fresh meeting summary.

Step 5 — Prompt construction

```
[system]
```

```
You are a helpful assistant with access to a personal knowledge base and  
Time Task Tracker (TTT) database of logged work entries.
```

```
Answer the user's question using ONLY the context passages provided below  
...
```

```
Context:
```

```
{ttt_rows}
```

```
[1] filename.txt (chunk 2, date 2026-08-16, score 0.821):
```

```
<chunk text>
```

```
[2] ...
```

```
[user turn N-2] ...
```

```
[assistant turn N-1] ...
```

```
[user] <current question>
```

Conversation history is appended as alternating `user` / `assistant` messages for multi-turn context. Retrieval always uses only the latest question so search stays focused.

Step 6 — LLM call

`get_provider().chat(messages)` dispatches to the configured provider. Temperature 0 is used for deterministic grounded answers.

Step 7 — Response

Returns `ChatResponse(answer: str, sources: list[dict])` where each source contains `source`, `score`, and `chunk_index`.

9. LLM provider abstraction

File: [backend/kb/llm.py](#)

All providers implement `BaseLLMProvider.chat(messages: list[dict]) -> str`.

Messages follow the OpenAI format: `[{"role": "system"|"user"|"assistant", "content": "..."}]`.

OllamaProvider

Posts to `{OLLAMA_HOST}/api/chat` with `stream=False`. Returns `response["message"]` `["content"]`.

OpenAIProvider

Posts to `{OPENAI_BASE_URL}/chat/completions`. Compatible with any OpenAI-spec endpoint — OpenAI, Groq, GLM-4 (ZhipuAI), Anyscale, Together, etc. Returns `choices[0].message.content`.

WatsonxProvider

Two-step per call:

1. **IAM token exchange** — POST `https://iam.cloud.ibm.com/identity/token` with `grant_type=apikey` → `access_token`
2. **Inference** — POST `{WATSONX_URL}/ml/v1/text/chat?version=2023-05-29` with token in `Authorization` header

Parameters: `temperature=0`, `max_tokens=600`, `frequency_penalty=0`, `presence_penalty=0`, `top_p=1`.

Note: The IAM token is fetched fresh on every call (no cache). Each call adds ~200–400 ms for the token exchange round-trip.

Adding a new provider

1. Subclass `BaseLLMProvider` in `llm.py` and implement `chat()`
2. Register it in `get_provider()` with a string key
3. Set `LLM_PROVIDER=<key>` in `.env`

10. Meeting intelligence pipeline

Files: [backend/api.py](#) · [backend/kb/extractors.py](#) · [backend/kb/pusher.py](#)

This pipeline is split across two HTTP requests to avoid the 30-second response timeout on Render free-tier (the LLM call alone takes 15–25 s). The same split also benefits OpenShift ingress timeout settings.

Request 1 — POST /ingest-meeting

1. Saves the uploaded file to a temp path

2. Calls `ingest(tmp_path, force=force, source_name=file.filename, user_id=user_id)`
3. Deletes the temp file
4. Returns `{status: "ok", filename: "..."} immediately (seconds)`

The file is now in pgvector.

Request 2 — `POST /summarize-meeting`

Receives `{filename, project_code, organizer, attendees}` as JSON.

1. **Metadata lookup** — calls `kb_search(filename, top_k=1, source_filter=filename, user_id=user_id)` to extract `doc_metadata`
2. **RAG summarization** — calls `kb_ask(summary_question, source_filter=filename, top_k=3, skip_ttt=True, user_id=user_id)`. `source_filter` scopes retrieval to only this file's chunks
3. **Time parsing** — `_parse_time_range(meeting_time, entry_date)` in `pusher.py` parses the raw time string into two naive `datetime` objects
4. **TTT push** — `push_meeting_entry(...)` inserts into `time_entries`
5. Returns `IngestMeetingResponse` with LLM summary, source citations, TTT entry ID, and any push error

Field sourcing priority

TTT field	Source priority
<code>meeting_title</code>	<code>doc_metadata.meeting_title</code> → filename
<code>entry_date</code>	<code>doc_metadata.meeting_date</code> → today
<code>start_time</code> / <code>end_time</code>	parsed from <code>doc_metadata.meeting_time</code> → null
<code>duration_minutes</code>	<code>doc_metadata.duration_minutes</code> → parsed from LLM summary → 60
<code>project_code</code>	request body → <code>doc_metadata.project_code</code> → parsed from LLM summary → filename stem
<code>organizer</code>	request body → <code>doc_metadata.organizer</code> → null
<code>attendees</code>	request body → <code>doc_metadata.attendees</code> → null
<code>billable</code>	<code>doc_metadata.billable</code> → false

TTT field	Source priority
description	LLM-generated summary

11. Time Task Tracker — write path

File: [backend/kb/pusher.py](#)

`push_meeting_entry()` uses a direct `psycopg2` connection to the TTT database. The `INSERT ... ON CONFLICT (id) DO NOTHING` pattern makes duplicate pushes idempotent — the same UUID will not overwrite an existing entry.

Time range parsing

`_parse_time_range(time_str, entry_date)` handles formats including:

- `10:00 AM – 10:30 AM CST`
- `14:00 – 14:45`
- `2:00 PM – 2:45 PM`

The regex `(\d{1,2}:\d{2})\s*(AM|PM)?` finds all time tokens. The first becomes `start_time`, the second becomes `end_time`. Both are constructed as naive `datetime` objects (no `tzinfo`) so the TTT displays wall-clock time as written.

Duration parsing

`_parse_duration_minutes(text)` applies two regexes to the LLM summary or the raw header value:

- `(\d+)\s*(?:hour|hr|h)\b` → multiply by 60
- `(\d+)\s*(?:minute|min|m)\b` → add directly

Defaults to 60 minutes if nothing is found.

12. Time Task Tracker — read path (chat context)

File: [backend/kb/ttt.py](#)

`query_ttt(question, user_id, limit, force_meetings)` is called from [chat.py](#) when TTT intent is detected. It classifies the question into one of four SQL shapes:

Shape	Trigger	Query
Meeting list	<code>force_meetings=True</code> or meeting history pattern	<code>SELECT ... WHERE user_id=? AND task_type='meeting' ORDER BY entry_date DESC</code>
Aggregated totals	"total", "sum", "how many hours"	<code>SELECT SUM(duration_minutes), COUNT(*) GROUP BY project_code, task_type</code>
Billable filter	"billable"	<code>SELECT ... WHERE user_id=? AND billable = TRUE</code>
Default recent	fallback	<code>SELECT ... WHERE user_id=? ORDER BY entry_date DESC</code>

All shapes support:

- **Date range** — extracted from "today", "this week", "last month", etc. Defaults to ± 365 days (wide window to handle future-dated entries such as upcoming meetings)
- **Project filter** — `ILIKE '%project%'` extracted from "for Honda", "on Honda", "Honda meeting"
- **Count** — extracted from "last two meetings", "last 3 entries" — overrides the default `limit`

Results are formatted as a readable text block prepended with `[Time Task Tracker – N result(s), date to date]` and injected ahead of vector chunks in the RAG prompt.

13. TTT REST API

File: [backend/ttt_api.py](#)

Mounted at `/ttt`. All routes require a valid Supabase JWT and scope every query to `user_id`.

Route	Method	Description
<code>/ttt/entries</code>	GET	List entries with optional <code>start_date</code> , <code>end_date</code> , <code>project_code</code> filters
<code>/ttt/entries</code>	POST	Create a new entry (returns 201)
<code>/ttt/entries/bulk-delete</code>	POST	Delete a list of entries by ID
<code>/ttt/entries/{id}</code>	GET	Fetch a single entry

Route	Method	Description
/ttt/entries/{id}	PUT	Update an entry (partial update)
/ttt/entries/{id}	DELETE	Delete an entry (204)
/ttt/summary	GET	Aggregated totals grouped by project and task type for a date range
/ttt/export/csv	GET	Download entries as CSV for a date range
/ttt/import/csv	POST	Bulk-import entries from a CSV file
/ttt/import/ics	POST	Bulk-import entries from an ICS calendar file
/ttt/projects	GET	List all distinct project codes for the user
/ttt/classify	POST	Infer project code and billable flag from a meeting title

CSV import format

Accepted column names are flexible (case-insensitive, normalised). Minimum required: a date column and a duration column. Optional: `meeting_title`, `project_code`, `task_type`, `billable`, `description`, `organizer`, `attendees`.

ICS import

Parses `VEVENT` blocks. `DTSTART` / `DTEND` provide start time, end time, and date. `SUMMARY` maps to `meeting_title`. Duration is computed from `DTEND` - `DTSTART`. The LLM `/ttt/classify` endpoint is **not** called during ICS import — project codes default to `GENERAL` and billable defaults to `false` unless the title matches built-in keyword heuristics.

AI classification (`/ttt/classify`)

`_classify(title, organizer)` applies regex patterns to infer:

- **Project code** — keyword patterns extracted from common project name formats
- **Billable** — matched against lists of billable keywords (`client`, `customer`, `consulting`) and non-billable keywords (`internal`, `team`, `admin`, `training`)

14. Frontend architecture

Files: [frontend/src/](#)

Single-page React application built with Vite. All API communication goes through [frontend/src/api.js](#) (KB routes) and [frontend/src/tttApi.js](#) (TTT routes). Both attach `Authorization: Bearer {token}` to every request.

Auth flow

[frontend/src/context/AuthContext.jsx](#) wraps the app in a `AuthProvider`. On mount it calls `supabase.auth.getSession()` and then subscribes to `onAuthStateChange` to keep the session in sync across tabs, token refreshes, and logouts. The `useAccessToken()` hook is the standard way to get the current JWT for API calls.

If the session is `null` (logged out), [App.jsx](#) renders [LoginPage.jsx](#) instead of the main tabs.

Login page

[LoginPage.jsx](#) supports:

- **Magic link** — `supabase.auth.signInWithOtp({ email })` — sends a login link to the user's email
- **Google OAuth** — `supabase.auth.signInWithOAuth({ provider: "google" })`
- **GitHub OAuth** — `supabase.auth.signInWithOAuth({ provider: "github" })`

KB components

Component	Responsibility
<code>Chat.jsx</code>	Multi-turn conversation UI. Maintains <code>history</code> array locally. Renders user/assistant bubbles, collapsible source citations, thinking state.
<code>Search.jsx</code>	Semantic search form with file type filter, source substring filter, top-K control. Snippet cards with expand-to-full-chunk toggle.
<code>Upload.jsx</code>	react-dropzone multi-file queue. Per-file upload status, force re-index checkbox.
<code>MeetingUpload.jsx</code>	Two-phase upload: calls <code>/ingest-meeting</code> (fast), then <code>/summarize-meeting</code> (slow LLM). Shows step indicators. Displays summary, source chunks, TTT entry ID, and TTT push errors.
<code>Sources.jsx</code>	Table of all indexed sources with chunk counts. Per-row delete with confirmation.

TTT components

Component	Responsibility
<code>TTTDashboard.jsx</code>	Stat cards (total hours, meeting count, billable hours) and bar chart of hours by project for a selected date range
<code>TTTEntries.jsx</code>	Filterable, sortable list of time entries. Inline edit, per-row delete, bulk delete with checkbox selection.
<code>TTTManualEntry.jsx</code>	Form to create a new entry with project code, task type, date, duration, billable flag, start/end time, and notes
<code>TTTImport.jsx</code>	react-dropzone zones for CSV import and ICS calendar import. Shows imported count and failure count.
<code>TTTReports.jsx</code>	Date-range report generation (summary grouped by project + task type). CSV export download.

API layer

```
// KB API – api.js
uploadFile(file, force, token)           // POST /upload
searchDocs({ query, top_k, ... }, token) // POST /search
chatWithKB({ question, history, ... }, token) // POST /chat
getSources(token)                        // GET /sources
deleteSource(source, token)              // DELETE /sources?source=...
ingestMeeting({ file, force }, token)    // POST /ingest-meeting
summarizeMeeting({ filename, ... }, token) // POST /summarize-meeting

// TTT API – tttApi.js
getEntries({ startDate, endDate, projectCode }, token) // GET /ttt/entr
createEntry(entry, token)                             // POST /ttt/ent
updateEntry(id, updates, token)                       // PUT /ttt/entr
deleteEntry(id, token)                                // DELETE /ttt/e
bulkDeleteEntries(ids, token)                         // POST /ttt/ent
getSummary({ startDate, endDate }, token)             // GET /ttt/summ
exportCSV(token, startDate, endDate)                  // GET /ttt/expo
importCSV(file, token)                                // POST /ttt/imp
importICS(file, token)                                // POST /ttt/imp
getProjects(token)                                    // GET /ttt/proj
classifyMeeting(title, organizer, token)              // POST /ttt/cla
```

`throwApiError(res)` is a shared helper in both API files that parses FastAPI error responses — tries `res.json().detail` first, falls back to `res.text()`.

15. OpenShift deployment

Files: [openshift/](#)

Key files

File	Purpose
<code>deploy.sh</code>	One-command deploy: build multi-arch images (amd64 + arm64), push to registry, log into cluster, apply secrets, deploy pods, wait for <code>Running</code> , print URL
<code>dump.sh</code>	Dump both databases to <code>openshift/backup.sql</code> before cluster expiry
<code>deploy.env.example</code>	Template — copy to <code>deploy.env</code> and fill in secrets
<code>postgres.yaml</code>	pgvector <code>StatefulSet</code> + Ceph RBD <code>PersistentVolumeClaim</code>
<code>backend.yaml</code>	FastAPI <code>Deployment</code> + <code>ClusterIP Service</code>
<code>frontend.yaml</code>	nginx <code>Deployment</code> + <code>Service</code> + OpenShift <code>Route (HTTPS)</code>
<code>secret.yaml</code>	Secret template (reference only — do not commit with values)
<code>Dockerfile.backend</code>	FastAPI image (built from <code>./backend</code>)
<code>Dockerfile.frontend</code>	Vite build → nginx image; <code>VITE_API_URL=/api</code> baked in
<code>nginx.conf</code>	Serves static files on port 8080, proxies <code>/api/*</code> to backend <code>ClusterIP</code>

Cluster migration workflow

```
# Before expiry — dump both DBs
./openshift/dump.sh      # writes openshift/backup.sql

# Update credentials in deploy.env
OC_SERVER=https://api.new-cluster.com:6443
OC_TOKEN=sha256~new-token
```

```
# Deploy to new cluster – backup.sql is auto-restored
./openshift/deploy.sh
```

`deploy.sh` detects `openshift/backup.sql` and runs `psql` restore inside the new postgres pod before the backend starts.

Post-deploy step

After each new cluster deploy, update **Supabase** → **Authentication** → **URL Configuration**:

- **Site URL** → `https://<route-host>`
- **Redirect URLs** → `https://<route-host>/**`

Without this, login redirects will fail.

16. File reference

Configuration

Path	Description
<code>.env</code>	Primary local config
<code>.env.example</code>	Template for <code>.env</code>
<code>openshift/deploy.env</code>	OpenShift secrets (gitignored)
<code>openshift/deploy.env.example</code>	Template for <code>deploy.env</code>
<code>recorder/.env</code>	Recorder settings — <code>KB_INGEST_URL</code> , <code>WHISPER_MODEL</code> , <code>AUDIO_DEVICE</code> , <code>RECORDINGS_DIR</code>
<code>frontend/.env.local</code>	Vite env — <code>VITE_API_URL</code> , <code>VITE_SUPABASE_URL</code> , <code>VITE_SUPABASE_ANON_KEY</code>

Core library — `backend/kb/`

File	Description
<code>config.py</code>	Loads <code>.env</code> , exports all config constants

File	Description
<code>auth.py</code>	Supabase JWT validation — ES256 (JWKS) + HS256 (static secret), returns <code>user_id</code>
<code>db.py</code>	SQLAlchemy engine, <code>SessionLocal</code> , <code>Document</code> ORM model, <code>init_db()</code>
<code>embedder.py</code>	<code>embed(text) → list[float]</code> — Nomic or Ollama provider
<code>extractors.py</code>	File-type dispatch → 800-char / 100-overlap chunking + meeting metadata extraction
<code>ingest.py</code>	Orchestrates discovery → dedup → extract → embed → upsert, with user isolation
<code>search.py</code>	Cosine search, snippet extraction, <code>list_sources</code> , <code>delete_source</code>
<code>chat.py</code>	RAG pipeline: intent classify → retrieve → re-rank → TTT inject → prompt → LLM
<code>llm.py</code>	<code>BaseLLMProvider</code> ABC → <code>OllamaProvider</code> / <code>OpenAIProvider</code> / <code>WatsonxProvider</code>
<code>pusher.py</code>	Writes meeting summaries to <code>time_entries</code> via direct psycopg2
<code>ttt.py</code>	Reads TTT rows for RAG context injection
<code>cli.py</code>	Click CLI: <code>kb init</code> · <code>kb ingest <path></code> · <code>kb search <query></code> · <code>kb list</code> · <code>kb delete <source></code>

Backend

File	Description
<code>backend/api.py</code>	FastAPI app — KB routes: <code>POST /upload</code> , <code>POST /search</code> , <code>GET /sources</code> , <code>DELETE /sources</code> , <code>POST /chat</code> , <code>POST /ingest-meeting</code> , <code>POST /summarize-meeting</code>
<code>backend/ttt_api.py</code>	TTT router mounted at <code>/ttt</code> — full CRUD, import, export, summary, classify

Frontend

File	Description
<code>frontend/src/App.jsx</code>	Root component — login gate, 4 KB tabs + 5 TTT tabs
<code>frontend/src/api.js</code>	KB fetch wrappers
<code>frontend/src/tttApi.js</code>	TTT fetch wrappers
<code>frontend/src/supabaseClient.js</code>	Supabase JS client initialised from <code>VITE_SUPABASE_URL</code> + <code>VITE_SUPABASE_ANON_KEY</code>
<code>frontend/src/context/AuthContext.jsx</code>	Session provider, <code>useSession()</code> , <code>useAccessToken()</code>
<code>frontend/src/components/LoginPage.jsx</code>	Magic link + Google + GitHub sign-in

Recorder (optional, macOS only)

File	Description
<code>recorder/teams_recorder.py</code>	ffmpeg + BlackHole audio capture → Whisper transcription → POST /upload

Output saved to `~/recordings/` as `teams_YYYYMMDD_HHMMSS.wav` + `.txt`.